

Image Steganography Using Steghide: Pre- and Post-Embedding Metadata and Hash Analysis

Aim

To hide a secret message inside an image using Steghide and analyze the image's metadata and hash values before and after the steganography process.

Objective

- To perform image steganography using Steghide in Kali Linux.
- To analyze image metadata before and after embedding.
- To compare MD5 and SHA-256 hash values before and after steganography.

Abstract

This practical demonstrates image steganography using **Steghide** in Kali Linux. Metadata and cryptographic hash values (MD5 and SHA-256) were collected before and after embedding a secret message into a JPEG image. The results showed that although the image appeared visually unchanged, the file size and hash values changed, confirming successful data embedding.

Tools Used

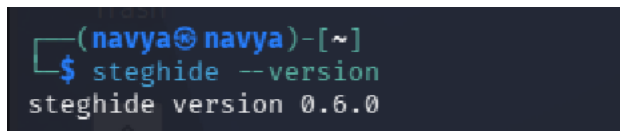
- Kali Linux
- Steghide
- ExifTool
- MD5
- SHA-256

Step 1: Verify Steghide Installation

To verify whether the Steghide tool is installed on the system.

Command

```
steghide -version
```



```
(navya@navya)-[~]  
$ steghide --version  
steghide version 0.6.0
```

If not installed: `sudo apt install steghide`

Step 2: Verify ExifTool Installation

To verify that ExifTool is installed.

Command

```
exiftool -ver
```

If not installed: `sudo apt install libimage-exiftool-perl`

```
(navya@navya)-[~/Steganography]
$ exiftool -ver
13.55
```

Step 3: Create a Working Directory

To create a dedicated folder for storing all files related to the steganography experiment.

Command

```
mkdir Steganography
```

Move into the directory:

Command

```
cd Steganography
```

Verify the current location:

Command

```
pwd
```

```
(navya@navya)-[~]
$ mkdir Steganography

(navya@navya)-[~]
$ cd Steganography

(navya@navya)-[~/Steganography]
$ pwd
/home/navya/Steganography
```

To confirm that the selected image is in JPEG format, which is supported by Steghide.

Verify the image:

Command

```
file flower.jpg
```

```
(navya@navya)-[~/Steganography]
$ file flower.jpg
flower.jpg: JPEG image data, JFIF standard 1.01, resolution (DPCM), density 56x56, segment length 16, baseline, precision 8, 1918x973, components 3
```

Step 4: View Image Metadata

To examine the metadata of the original image before embedding any secret data.

Command

```
exiftool flower.jpg
```

```
(navya@navya)-[~/Steganography]
$ exiftool flower.jpg
ExifTool Version Number      : 13.55
File Name                    : flower.jpg
Directory                    : .
File Size                    : 402 kB
File Modification Date/Time   : 2026:07:11 10:54:25+05:30
File Access Date/Time        : 2026:07:11 10:54:35+05:30
File Inode Change Date/Time   : 2026:07:11 10:54:25+05:30
File Permissions              : -rw-rw-r--
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
JFIF Version                 : 1.01
Resolution Unit              : cm
X Resolution                 : 56
Y Resolution                 : 56
Image Width                  : 1918
Image Height                 : 973
Encoding Process              : Baseline DCT, Huffman coding
Bits Per Sample              : 8
Color Components              : 3
Y Cb Cr Sub Sampling         : YCbCr4:4:4 (1 1)
Image Size                   : 1918x973
Megapixels                   : 1.9
```

Metadata contains information about the image, including:

- File Name
- File Size
- Image Dimensions
- MIME Type
- Modification Date
- Compression Type

This information serves as a baseline for comparison after steganography.

Step 5: Generate MD5 Hash

To generate the MD5 checksum of the original image.

Command

```
md5sum flower.jpg
```

```
(navya@navya)-[~/Steganography]
$ md5sum flower.jpg
74b9798d6d91e158bf26211ed39d7870 flower.jpg
```

Step 6: Generate SHA-256 Hash

To generate the SHA-256 hash of the original image.

Command

```
sha256sum flower.jpg
```

```
(navya@navya)-[~/Steganography]
$ sha256sum flower.jpg
541c4969a8dec27defb251d76c76b5ac1cb2bb0535b38368de2215127d18bda1 flower.jpg
```

Step 7: Create the Secret Message

Command

```
nano secret.txt
```

To create a text file containing the confidential message that will be embedded inside the image.

Example:

```
This is a confidential message.
```

```
Prepared by: Navya Sree
```

```
Cyber Security Laboratory
```

Save using:

- Ctrl + O
- Enter
- Ctrl + X

Verify:

```
cat secret.txt
```

```
(navya@navya)-[~/Steganography]
$ nano secret.txt

(navya@navya)-[~/Steganography]
$ cat secret.txt
This is a confidential message.

Prepared by:
K Navya Sree
Cyber Security Lab
```

Step 8: Embed the Secret Message

To hide the secret message inside the JPEG image.

Command

```
steghide embed -cf flower.jpg -ef secret.txt
```

Explanation

Steghide embeds the contents of secret.txt into flower.jpg. During this process, the user is prompted to enter a passphrase, which is required later to extract the hidden data.

Example:

Enter passphrase:

Re-enter passphrase:

embedding "secret.txt" in "flower.jpg"... done

```
(navya@navya)-[~/Steganography]
$ steghide embed -cf flower.jpg -ef secret.txt
Enter passphrase:
Re-Enter passphrase:
embedding "secret.txt" in "flower.jpg" ... done
```

Step 9: Verify Embedded Information

To verify whether the image contains embedded data.

Command

steghide info flower.jpg

```
(navya@navya)-[~/Steganography]
$ steghide info flower.jpg
"flower.jpg":
  format: jpeg
  capacity: 20.8 KB
Try to get information about embedded data ? (y/n) y
Enter passphrase:
  embedded file "secret.txt":
    size: 78.0 Byte
  sf_encrypted: rijndael-128, cbc
  compressed: yes
```

Step 10: Extract the Secret Message

To recover the hidden message from the stego image.

Command

```
steghide extract -sf flower.jpg
```

After entering the correct passphrase, Steghide extracts the embedded file back into the current directory.

Verify:

```
cat secret.txt
```

```

(navya@navya)-[~/Steganography]
$ steghide extract -sf flower.jpg
Enter passphrase:
the file "secret.txt" does already exist. overwrite ? (y/n) y
wrote extracted data to "secret.txt".

(navya@navya)-[~/Steganography]
$ cat secret.txt
This is a confidential message.

Prepared by:
K Navya Sree
Cyber Security Lab

```

Step 11: Generate MD5 Hash After Steganography

To calculate the MD5 hash after embedding the secret data.

Command

```
md5sum flower.jpg
```

```

(navya@navya)-[~/Steganography]
$ md5sum flower.jpg
6c60603b31efba45362ed7e93c7331f3 flower.jpg

```

Since Steghide modifies the image file by embedding hidden information, the file contents change. Consequently, the MD5 hash also changes.

Step 12: Generate SHA-256 Hash After Steganography

To calculate the SHA-256 hash after steganography.

Command

```
sha256sum flower.jpg
```

```

(navya@navya)-[~/Steganography]
$ sha256sum flower.jpg
5ae09c42bb93e3fa9c7a17ba51d34f3d14f4b385367176924b66524e19a0ab0a flower.jpg

```

The SHA-256 value changes because the binary content of the image has been modified.

Step 13: Compare Metadata

To compare the image metadata after embedding the secret message.

Command

```
exiftool flower.jpg
```

```
(navya@navya)-[~/Steganography]
$ exiftool flower.jpg
ExifTool Version Number      : 13.55
File Name                    : flower.jpg
Directory                    : .
File Size                    : 425 kB
File Modification Date/Time   : 2026:07:11 10:57:32+05:30
File Access Date/Time        : 2026:07:11 10:57:47+05:30
File Inode Change Date/Time   : 2026:07:11 10:57:32+05:30
File Permissions              : -rw-rw-r--
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
JFIF Version                 : 1.01
Resolution Unit               : cm
X Resolution                  : 56
Y Resolution                  : 56
Image Width                  : 1918
Image Height                  : 973
Encoding Process              : Baseline DCT, Huffman coding
Bits Per Sample               : 8
Color Components              : 3
Y Cb Cr Sub Sampling         : YCbCr4:4:4 (1 1)
Image Size                   : 1918x973
Megapixels                   : 1.9
```

Metadata Analysis

- Image format remained **JPEG**.
- Image resolution remained **1918 × 973 pixels**.
- File size increased from **402 KB** to **425 KB** after embedding.
- File modification timestamp changed after embedding.
- Other image properties remained unchanged.

Hash Analysis

- The **MD5** hash value changed after embedding the secret message.
- The **SHA-256** hash value also changed after embedding.
- The changed hash values confirm that the binary content of the image was modified while preserving its visual appearance.

Observations

- The image appearance remained unchanged after steganography.
- The secret message was successfully embedded into the image.
- Metadata showed only minor changes, mainly in file size and modification time.
- MD5 and SHA-256 hash values changed after embedding.
- The hidden message was successfully extracted using the correct passphrase.

Conclusion:

The practical successfully demonstrated image steganography using Steghide in Kali Linux. The secret message was embedded and extracted successfully, while metadata and hash analysis confirmed changes in the image's binary data without affecting its visual appearance.